

are planning to manage some cloud environment like AWS EC2, then dynamic inventory is the way to go. The Ansible project provides some out-of-the-box scripts to work with dynamic inventories for cloud projects like AWS EC2, Rackspace Cloud, OpenStack, etc. The default inventory file for Ansible is `/etc/ansible/hosts`, but you could use any desired location with the option `-i` with the Ansible command line or by tuning the hostfile in `ansible.cfg`. There are a number of ways in which you could put the hosts' information in the inventory file but shown below is the basic format of the inventory, which clubs various hosts into different logical groups based upon their functionalities:

```
[webservers]
<WEBSERVER1>
<WEBSERVER2>
...

[dbservers]
<DBSERVERS1>
<DBSERVERS2>
...
```

The advantage of grouping different hosts is that you could orchestrate anything on all the hosts under a group, just by supplying the group name to Ansible, which defines the functionalities you want to execute on remote hosts as modules. You need to mention the desired module to run Ansible. The following command is the way you could test whether all your Web servers are responding:

```
ansible webservers -m ping -i /usr/local/etc/ansible/hosts
-u <LOGIN> --private-key <PRIVATE KEY>
```

The following Ansible command transfers a shell script to all your servers and executes it on all the hosts in parallel:

```
ansible all -m script -a "<YOUR SHELL SCRIPT>" -i /usr/
local/etc/ansible/hosts -u <LOGIN> --private-key <PRIVATE
KEY>
```

The command to collect the information about your hosts, known as facts, is:

```
ansible all -m setup -i /usr/local/etc/ansible/hosts -u
<LOGIN> --private-key <PRIVATE KEY>
```

You could reboot all your RedHat based `dbserver` hosts using the following command:

```
ansible dbservers -s -m shell -a "/sbin/reboot" -i /usr/
local/etc/ansible/hosts -u <SUDO LOGIN> --private-key
<PRIVATE KEY>
```

The above single-line commands are just an ad hoc way to use Ansible, and it comes prepackaged with a lot of modules covering everything from a simple ping to complicated cloud based provisioning. Ansible takes a description for the workflows containing various activities and orchestration in a YAML file format, and then executes that to bring about the desired result. These YAML descriptions containing instructions for Ansible to execute are known as playbooks, which are a standard way of accomplishing anything through Ansible in a single shot. It's highly recommended that you start any multi-step workflow by creating the playbooks. These are meant to be stored in your version control system to share and reuse, once you verify them for the correct workflows.

Ansible also reads its global settings from a config file named `ansible.cfg` (highly commented example of a config file link is provided in the Reference section at the end of the article) and that could save your typing when using the Ansible and Ansible-playbook commands. For example, if you put the following settings in `ansible.cfg` then you don't need to mention these again and again while using the Ansible and Ansible-playbook commands:

```
[default]
...
hostfile          = <Host file path>
host_key_checking = False
log_path          = <Dir to contain ansible.log>
private_key_file  = <Private key file path>
...
```

With the basic settings in the `ansible.cfg`, you could run any Ansible playbook simply by typing (`-vvvv` option allows the verbose output useful for debugging):

```
ansible-playbook <name of the playbook> -vvvv
```

Now let's explore some fundamentals of how to create the Ansible playbooks. A playbook defines various tasks that are to be performed over a group of remote hosts defined in the Ansible inventory. The combination of hosts and related tasks together is known as a 'play' in Ansible terminology. A playbook could contain multiple plays in case you want to perform different tasks on different hosts. By default, the tasks in a play are performed in parallel over the hosts defined by the respective groups in the inventory. But Ansible also provides the constructs through which you could instruct to run the tasks over hosts in serial manner. The following example presents a playbook, which runs operating systems-specific scripts in parallel on different groups of hosts defined in the inventory:

```
<start of code>
```